TECHNICAL HIRING ASSESSMENT

Data Science Trainer

Python Logic · Data Analysis with pandas · Machine Learning · Deep Learning

Level	Medium to High (industry / senior-trainer standard)
Total marks	100 (Section A 20 · Task 1 25 · Task 2 25 · Task 3 20 · Viva & Teach-back 10)
Format	Section A: 120-minute proctored round | Tasks 1–3: take-home, 72 hours | Viva: 45 minutes
AI tools	**STRICTLY PROHIBITED** for every part of this assessment (see Section 2)
Datasets supplied	Task 1: 4 files (≈ 252,000 transaction rows) | Task 2: 3 files (40,000 loan applications) | Task 3: public dataset

Candidate name	
Date issued	
Submission deadline (date & time)	
Candidate signature	

Read this entire document before you start. Questions about the *wording* of a task may be emailed to the assessment coordinator; no help will be given on *how* to solve a task. Where something is not specified, make a sensible, documented assumption. Documenting your assumptions is itself part of the assessment.

1. Assessment overview

This assessment simulates the work of a data science trainer, who must be able to **do** real data science and **teach** it. You will be judged on correctness, rigour, code quality, how clearly you communicate, and whether your work could be used as teaching material for students.

#	Section	What you will do	Mode / time	Marks
A	Python logic round	11 problems: palindromes, number theory, strings, matrices, debugging / output prediction	Proctored, offline, 120 min	20
1	Data analysis & PDF report	Clean a large, multi-source, dirty retail dataset using **pandas only**, answer 15 business questions, auto-generate a PDF report	Take-home	25
2	Machine learning	Clean a dirty loan dataset, build a leak-free pipeline, compare 9+ models, tune, choose a threshold, write test cases, predict on a hidden test set	Take-home	25
3	Deep learning	Train, evaluate and compare MLP and CNN models on a public image dataset; predict on new images	Take-home	20
V	Viva & teach-back	Explain and live-modify your own code; deliver a 10-minute mini-lesson	Live, 45 min	10

Suggested schedule for the 72-hour take-home

Day	Recommended focus
Day 1	Task 1 – ingestion, data-quality audit, cleaning, validation, analysis
Day 2	Task 1 – report generation · Task 2 – cleaning, EDA, pipeline, model comparison
Day 3	Task 2 – tuning, threshold, tests, predictions · Task 3 – deep learning · README & final checks

Pass mark: **65 / 100**, with at least **50 %** in every section. A score below 50 % in the viva on your own code voids the take-home score (see Section 2).

2. Ground rules & academic integrity

NO ARTIFICIAL INTELLIGENCE, IN ANY FORM
You may **not** use any generative-AI or AI-assisted coding tool at any stage, including (but not limited to): ChatGPT, Claude, Gemini, Copilot, Cursor, Codeium / Windsurf, Tabnine, Amazon Q / CodeWhisperer, JetBrains AI, Google Colab AI / Gemini in Colab, Kaggle AI assistants, Replit AI, Perplexity, and AI features built into IDEs, notebooks or browsers. **Turn off AI autocomplete in your editor before you start.**
Using AI for any part of the work, including "just explaining an error", means immediate disqualification.

What IS allowed

- Official documentation: docs.python.org, pandas.pydata.org, numpy.org, matplotlib.org, scikit-learn.org, tensorflow.org / keras.io, pytorch.org.
- Your own notes, books and previously written code (you must be able to explain every line of it).
- Reading Q&A forums for *syntax*. Copying solution code from a forum, blog, Kaggle notebook or GitHub repository is plagiarism.
- Standard, non-AI editor features: syntax highlighting, linting, plain (non-AI) autocomplete.

How integrity will be verified

- **Section A** runs offline, with your screen and webcam shared and the internet disconnected.

- **Git history:** all take-home work must live in one git repository, with at least **15 meaningful commits** spread over the work period. Do not squash commits. Submit the whole `.git` folder.
- **Similarity check:** submissions are compared automatically (MOSS / JPlag-style token matching) with other candidates' work and with public notebooks.
- **Live viva:** you will be asked to explain random lines of your code and to change it live (for example, add a model, change a cleaning rule or re-bin a feature) within a few minutes.
- **Signed declaration:** fill in and sign Appendix C and include it in your submission.

General technical rules

- Python 3.10 or later. Include a `requirements.txt` with pinned versions. Everything must run from a clean environment with the commands in your README.
- Fix all random seeds (`random_state=42` or similar). Results must be reproducible.
- Never edit the supplied data files by hand (no Excel, no find-and-replace). Every transformation must be in code.
- Notebooks must be restarted and run top to bottom before submission, with outputs kept. Repeated logic belongs in `.py` modules with functions and docstrings.
- Write as if your work will be handed to students: use clear markdown explanations, meaningful names and no dead code.

3. Section A — Python logic round (20 marks)

Rules for Section A

- 120 minutes, proctored and offline. Submit a single file, `sectionA_solutions.py`.
- **No imports at all**: no math, itertools, collections, functools, re, numpy or sympy. Use built-ins only. (`time` may be imported only to time your own code.)
- Use exactly the function names and signatures given. Your file will be run automatically against **hidden test cases**, so the exact return values and types matter.
- Every function needs a docstring, a time/space-complexity comment and at least 3 `assert` tests inside `if __name__ == "__main__":`.
- Problem-specific restrictions (marked "Restriction") are mandatory. Breaking one scores zero for that problem.

A1 · Sentence palindrome (1 mark)

Return `True` if the string is a palindrome when you consider only letters and digits and ignore case.
Restriction: No slicing reversal (`[::-1]`), no `reversed()`, no building a cleaned copy of the string. Use the two-pointer technique: O(n) time, O(1) extra space.

```
def is_sentence_palindrome(s: str) -> bool

is_sentence_palindrome("A man, a plan, a canal: Panama")  -> True
is_sentence_palindrome("race a car")                      -> False
is_sentence_palindrome("")                                -> True
```

A2 · Longest palindromic substring + palindrome count (2 marks)

Return a tuple `(longest, count)`. *longest* is the longest palindromic substring; if two have the same length, return the one that appears **first**. *count* is the number of palindromic substrings, counted by position (so "aaa" has 6).
Restriction: The solution must be better than O(n^3) (expand-around-centre or dynamic programming).

```
def longest_palindrome(s: str) -> tuple

longest_palindrome("babad") -> ("bab", 7)
longest_palindrome("cbbd")  -> ("bb", 5)
longest_palindrome("aaa")   -> ("aaa", 6)
```

A3 · Next palindromic number (2 marks)

Return the smallest palindromic integer **strictly greater** than n (n ≥ 0).
Restriction: No brute-force increment-and-check. It must return instantly for 18-digit inputs. Think about mirroring the left half.

```
def next_palindrome(n: int) -> int

next_palindrome(808)   -> 818
next_palindrome(999)   -> 1001
next_palindrome(12345) -> 12421
```

A4 · Number-property toolkit (1.5 marks)

(a) `armstrong_numbers(limit)` returns a sorted list of all Armstrong (narcissistic) numbers in [1, limit]. A k-digit number is Armstrong if the sum of its digits, each raised to the power k, equals the number.
(b) `is_happy(n)` returns True if repeatedly replacing n with the sum of the squares of its digits reaches 1. You must detect the cycle when it does not.
Restriction: Digit extraction must use only `%` and `//`. No `str()` or `len(str())`. Print the **count** and the **list** of Armstrong numbers up to 1,000,000.

```
armstrong_numbers(500) -> [1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407]
is_happy(19) -> True        is_happy(4) -> False
```

A5 · Prime toolkit (2 marks)

(a) `sieve(n)` implements the Sieve of Eratosthenes and returns a list of booleans. Print how many primes are ≤ 1,000,000.
(b) `count_twin_primes(n)` returns the number of pairs (p, p+2), both prime, with p+2 ≤ n.
(c) `factorise(n)` returns the prime factorisation as a string in exactly the format shown.
(d) `goldbach(n)` returns, for an even n > 2, the pair (p, q) with p ≤ q, both prime, p + q = n and the **smallest** possible p.

```
factorise(360)  -> "2^3 x 3^2 x 5"
factorise(97)   -> "97"
goldbach(100)   -> (3, 97)
count_twin_primes(20) -> 4      # (3,5) (5,7) (11,13) (17,19)
```

A6 · Kaprekar's routine (1.5 marks)

For a 4-digit number (1000–9999), repeatedly subtract the ascending-digit number from the descending-digit number until you reach 6174. Return the number of steps. Keep leading zeros (2111 → 2111 − 1112 = 0999, which is treated as the digits 0, 9, 9, 9). Return −1 for repdigits (1111, 2222, ...). Then answer: across all numbers from 1000 to 9999, what is the **maximum** number of steps, and **how many** numbers need exactly that many?

```
def kaprekar_steps(n: int) -> int

kaprekar_steps(3524) -> 3
kaprekar_steps(6174) -> 0
kaprekar_steps(1111) -> -1
```

A7 · Number to words, Indian numbering system (2 marks)

Convert 0 ≤ n ≤ 99,99,99,999 to words using Crore, Lakh, Thousand and Hundred. Use title case, single spaces, no hyphens and no “and”.

```
def to_words_indian(n: int) -> str

to_words_indian(105)       -> "One Hundred Five"
to_words_indian(1234567)   -> "Twelve Lakh Thirty Four Thousand Five Hundred Sixty Seven"
to_words_indian(100000000) -> "Ten Crore"
```

A8 · Roman numerals with strict validation (2 marks)

`to_roman(n)` handles 1–3999. `from_roman(s)` must **raise ValueError** for any non-canonical or invalid numeral: invalid symbols, an empty string, or non-standard forms such as "IIII", "VX" or "IC". Think about how a round-trip check could help.

```
to_roman(1994)      -> "MCMXCIV"
from_roman("MMXXVI") -> 2026
from_roman("IIII")   -> raises ValueError
```

A9 · Collatz, perfect and amicable numbers (2 marks)

(a) `longest_collatz(limit)` returns `(start, chain_length)` for the start value below *limit* that produces the longest Collatz chain. Chain length counts every term, including the start and the final 1. **Restriction:** You must write your own memoisation with a dict (no lru_cache). Runtime for limit = 1,000,000 must be under 15 s.
(b) Print all **perfect numbers** below 10,000 and all **amicable pairs** below 10,000. Divisor sums must be computed in O(√n).

```
Chain for 13: 13 → 40 → 20 → 10 → 5 → 16 → 8 → 4 → 2 → 1   (length 10)
longest_collatz(10) -> (9, 20)
```

A10 · Strings & matrices (2 marks)

(a) `rle_encode(s)` / `rle_decode(s)`: run-length encoding. Counts may have several digits.
(b) `first_bracket_error(s)` returns the index of the first bracket error in a string containing `()[]{}` and other characters. A closing bracket that does not match gives its own index; if openers are left unclosed, return the index of the earliest unclosed opener. Return −1 if the string is balanced.
(c) `spiral(matrix)` returns the elements of an m × n list-of-lists in clockwise spiral order (no numpy).

```
rle_encode("aaabccdddd") -> "a3b1c2d4"      rle_decode("a3b1c2d4") -> "aaabccdddd"
first_bracket_error("{[()]}") -> -1   first_bracket_error("([)]") -> 2
spiral([[1,2,3,4],[5,6,7,8],[9,10,11,12]]) -> [1,2,3,4,8,12,11,10,9,5,6,7]
```

A11 · Trainer special: predict the output, explain, fix (2 marks)

For each snippet, **first write down the exact output without running it** (in a comment). Then explain the underlying concept in 2–3 sentences, as you would to a beginner, and show the corrected or idiomatic version. Copy all of this into `sectionA_solutions.py` as comments.

```
# S1
def add_item(item, bucket=[]):
    bucket.append(item)
    return bucket
print(add_item(1)); print(add_item(2))

# S2
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])

# S3
a = [[0] * 3] * 3
a[0][0] = 1
print(a)

# S4
print(0.1 + 0.2 == 0.3)

# S5
nums = [2, 4, 6, 8, 1]
for n in nums:
    if n % 2 == 0:
        nums.remove(n)
print(nums)

# S6
print(-7 // 2, -7 % 2, round(2.5), round(3.5), int(-2.7))

# S7
print(True + True, [] == False, bool([]) == False)

# S8
x = [1, 2, 3]; y = x; y += [4]; z = x[:]; z.append(5)
print(x, y, z)
```

4. Task 1 — Retail data analysis & automated PDF report (25 marks)

Scenario. *UrbanCart* is an Indian online retailer. The leadership team wants a data-driven review of the business from **1 January 2023 to 30 June 2025**. The data was exported from four different internal systems by different teams and **has not been cleaned**. Expect every kind of real-world problem. Finding them is part of the task, and no list of issues will be provided.

> **Library restriction for Task 1**
> All data loading, cleaning, transformation and analysis must use **pandas** (numpy is allowed only for numeric helpers such as `np.where` / `np.select`). Charts: **matplotlib** or `DataFrame.plot` only. PDF: `matplotlib.backends.backend_pdf.PdfPages`, **reportlab** or **fpdf2**.
> **Not allowed:** automated EDA/profiling tools (ydata-profiling, sweetviz, dtale, autoviz, lux), polars, SQL engines (duckdb, sqlite, pandasql), scikit-learn, seaborn, plotly, Excel/Power BI/Tableau, or any manual editing of the report.

4.1 Input files (folder: datasets/task1_retail/)

File	Format	Approx. size	Content
sales_transactions.csv	CSV	≈ 252,000 rows · 28 MB	One row per order line (an order can have several lines)
customers.csv	CSV (inspect the delimiter)	≈ 32,450 rows	Customer master from the CRM
products.json	JSON (list of records)	≈ 875 records	Product catalogue with list and cost prices
city_master.csv	CSV	34 rows	**Clean** reference list of cities with state, tier and region
			Column descriptions: Appendix A

4.2 Business definitions (use exactly these)

Metric	Definition
Line value	Qty × Unit Price × (1 − Discount). Shipping fee is **excluded** from all revenue metrics.
GMV	Sum of line values for all lines that are **not Cancelled**.
Net Revenue	GMV excluding lines with status **Returned**. Shipped and Delivered lines count.
Profit	Net Revenue − Σ(Qty × cost_price) for the same lines.
AOV	Net Revenue ÷ number of distinct orders that contribute to Net Revenue.
Return rate	Returned lines ÷ non-cancelled lines. Cancellation rate is measured at **order** level.

4.3 Part A: Ingestion, data-quality audit & cleaning (10 marks)

- **Ingest** all four files robustly with pandas. Report the shape and dtypes of every file as loaded.
- **Audit** every column of every file. Produce a `data_quality_log.csv` with columns: file, column, issue, rows_affected, example_values, action_taken, rationale. A strong submission identifies **at least 25 distinct issues**.
- **Clean** with documented rules: standardise headers, keys and categories (map cities to `city_master.csv`); parse every date and number format; remove exact and near duplicates; decide how to handle orphan keys, impossible values, outliers and logically inconsistent records. **Justify every decision**; do not just drop rows silently.
- Produce a **cleaning funnel** table showing row counts before and after each step for each file.

- **Validate**: write a function `validate(df)` with at least 12 `assert` checks run on the cleaned data (unique keys, value ranges, allowed categories, date window, referential integrity, no negative money, etc.). Save cleaned tables plus one merged analysis table to `outputs/cleaned/`.

4.4 Part B: Analysis, answer all 15 questions (10 marks)

Each answer needs the pandas code, a number or table, a chart where it makes sense, and a 1–3 sentence business interpretation. You are expected to use `groupby/agg`, `pivot_table`, `merge`, `resample`, `rolling`, `cut/qcut`, `crosstab`, `melt`, `shift`, `rank` and method chaining where suitable.

#	Question
Q1	Headline KPIs: GMV, Net Revenue, Profit, margin %, number of orders, AOV. Net Revenue growth 2024 vs 2023 (%).
Q2	Monthly Net Revenue trend with a 3-month rolling average. Which are the peak and lowest months, and why?
Q3	Find any anomalous period in daily order volume statistically (for example a rolling z-score). Report the dates and give a likely explanation.
Q4	Category scorecard: revenue share, profit margin and return rate. Which category is “revenue-rich but margin-poor”?
Q5	Return rate by category × discount band (0–10, 10–20, 20–30, 30–40, 40 %+). Do deeper discounts drive returns?
Q6	Cancellation rate by payment method (order level). Quantify the GMV lost to Cash-on-Delivery cancellations.
Q7	Top 10 cities by Net Revenue; average delivery days by city tier; the relationship between delivery days and rating (correlation + grouped means).
Q8	Customer concentration: % of Net Revenue from the top 20 % of customers. Build an **RFM** segmentation (quintile scores, named segments) and size each segment.
Q9	Monthly **cohort retention matrix** (cohort = month of first purchase), shown as a heatmap. Comment on retention after months 1, 3 and 6.
Q10	Age group (from DOB) × category spend pivot, and the gender split of revenue after cleaning gender.
Q11	Hour-of-day × day-of-week order heatmap. Busiest hour; weekend vs weekday average daily orders.
Q12	Quarterly share of Mobile App orders and of UPI payments. Describe the trends.
Q13	Coupon audit: the NEWUSER coupon is meant for a customer's first order only. Detect misuse, count the customers involved and estimate the discount value leaked.
Q14	Do loyalty tiers in the CRM match actual spend? Show spend distribution by tier and flag inconsistencies.
Q15	Basket analysis: average lines per order, and the top 10 category pairs bought together in the same order (self-merge on order id).

4.5 Part C: The automated PDF report (5 marks)

Running `python build_report.py` must regenerate `outputs/UrbanCart_Analysis_Report.pdf` from the raw files, end to end, with no manual steps. Requirements:

- Cover; a one-page **executive summary** (5 insights + 5 recommendations, each backed by a number).
- A data-quality section: issues found, the cleaning funnel table, and before/after examples.
- At least **12 charts**, each with a title, labelled axes with units, readable tick labels and an insight caption. No default unlabelled plots.
- Formatted tables (Indian number formatting, e.g. ₹1,25,40,000 or ₹1.25 Cr, used consistently); page numbers; a table of contents.
- An appendix of all assumptions and their likely impact. Length 12–30 pages, written for a non-technical leadership team.

5. Task 2 — Machine learning: loan default prediction (25 marks)

Scenario. *FinTrust*, a digital lender, wants a model that predicts at the **moment of application** whether an applicant will default (90+ days past due within 12 months). The model will be used to approve or reject applications automatically. The data has been pulled from several systems and is dirty.

File	Rows	Description
loan_train.csv	≈ 32,600	Historical applications with the target column `defaulted`
loan_test.csv	8,000	Hold-out applications **without** the target. You submit predictions; the evaluator scores them against hidden labels.
loan_scenario_cases.csv	12	Hand-crafted applicant profiles your final pipeline must process (Section 5.3)

Column descriptions are in **Appendix B**. **Business costs:** approving an applicant who then defaults (false negative) costs **₹1,00,000** on average; rejecting an applicant who would have repaid (false positive) costs **₹15,000** in lost interest income.

Allowed libraries: pandas, numpy, matplotlib/seaborn, scikit-learn, scipy, joblib, pytest, imbalanced-learn, and optionally xgboost / lightgbm / catboost.

5.1 Required steps

Step	Topic	What is expected
2A	Problem framing	Business objective, unit of prediction, which metrics you will use and why plain accuracy is not enough here, and the cost matrix.
2B	Cleaning & EDA	Parse every messy column (units, formats, booleans, categories, sentinels, structural missingness). Handle duplicates and target issues. Univariate and bivariate EDA with default rate by key features. **Decide which columns must not be used and explain why.**
2C	Feature engineering	At least 5 meaningful engineered features (for example EMI from the amortisation formula, a recomputed obligation-to-income ratio, loan-to-income, missingness flags, banding). Justify each one.
2D	Leak-free pipeline	Stratified train/validation split **before** any fitting. All imputation, scaling and encoding inside an sklearn `Pipeline` + `ColumnTransformer`. Handle high-cardinality and unseen categories.
2E	Model zoo (≥ 9 models)	DummyClassifier baseline, Logistic Regression, KNN, Naive Bayes, Decision Tree, Random Forest, Extra Trees, Gradient Boosting / HistGradientBoosting, AdaBoost, SVM (a subsample is acceptable), plus optionally XGBoost/LightGBM. For each model report Accuracy, Precision, Recall, F1, ROC-AUC, PR-AUC, fit time, and 5-fold stratified CV mean ± std, in one comparison table and chart.
2F	Class imbalance	Compare at least two strategies (e.g. none vs class_weight vs SMOTE inside the CV loop). Explain the effect on precision and recall.
2G	Tuning	GridSearchCV or RandomizedSearchCV on the best 2 models with a sensible search space. Report the best parameters, CV score and the gain over defaults.
2H	Threshold & cost	Choose the decision threshold that minimises expected business cost on validation data. Compare cost at 0.5 vs your threshold. Show ROC, PR and calibration curves.
2I	Interpretability	Global importance (permutation importance; coefficients for LR) plus a written explanation of individual predictions for TC01, TC02 and TC08.
2J	Final predictions	Retrain the chosen pipeline and predict `loan_test.csv`. Save `predictions_loan_test.csv` with exactly the columns `applicant_id, default_probability, default_pred` (8,000 rows; pred uses your chosen threshold).

5.2 Software test cases (pytest)

Create `tests/test_pipeline.py` with at least **12 passing tests**. They must include:

- Unit tests for each parsing function, e.g. income parsing of `'5.4 LPA'`, `'₹5,40,000'`, `'45000/month'` and `'540000'` → 540000; boolean parsing; loan term `'3 yrs'` → 36.
- The pipeline accepts unseen categories and missing values without crashing.
- Output shape and type; every probability lies in [0, 1]; predictions are deterministic across two runs.
- Invalid input (e.g. a negative loan amount or a non-numeric age such as 'abc') raises a clear `ValueError` or is flagged; it must not be silently scored.
- The saved model loads from disk and gives the same predictions as the in-memory model.

5.3 Scenario cases & deployment

- Run all 12 rows in `loan_scenario_cases.csv` through your **final saved** pipeline. Save `scenario_predictions.csv` and discuss in the notebook whether each result is sensible and why.
- Provide a CLI: `python predict.py --input any.csv --output preds.csv` that loads the saved model (joblib) and handles raw, dirty input in the same format as the supplied files.
- Minimum quality bar: validation ROC-AUC ≥ 0.80. Your hold-out score is computed by the evaluator.

6. Task 3 — Deep learning: image classification (20 marks)

Dataset: Fashion-MNIST: 70,000 grayscale 28 × 28 images of 10 clothing classes (60,000 train / 10,000 test). Load it in code. Do **not** download pre-processed or third-party versions.

Source	How to load
TensorFlow / Keras	`tf.keras.datasets.fashion_mnist.load_data()`
PyTorch	`torchvision.datasets.FashionMNIST(root, train=True/False, download=True)`
Public link (reference)	https://github.com/zalandoresearch/fashion-mnist

Use **one** framework (TensorFlow/Keras or PyTorch). A GPU is not required; every model below trains on a laptop CPU or on Google Colab with **Colab AI features switched off**.

6.1 Required steps

Step	Topic	What is expected
3A	EDA	Shapes, dtypes, class balance; a grid of 10 samples per class; pixel intensity statistics.
3B	Pre-processing	Normalisation, reshape, label encoding. A stratified 50,000 / 10,000 train/validation split from the training set. **The test set stays untouched until final evaluation.**
3C	Model 1 – MLP baseline	At least 2 hidden layers with dropout. Target: test accuracy ≥ **87 %**.
3D	Model 2 – CNN	At least 2 convolution blocks (Conv → BatchNorm → ReLU → Pool), dropout, dense head. Target: ≥ **91 %**.
3E	Model 3 – improved CNN	Data augmentation (small shifts, zoom, horizontal flip) and/or a deeper or residual design, plus a learning-rate schedule. Target: ≥ **92.5 %**.
3F	Training discipline	Fixed seeds, EarlyStopping, checkpointing the best validation model, ReduceLROnPlateau (or a scheduler). Log parameter count, epochs and wall-clock time. Plot training vs validation loss and accuracy curves for every model and diagnose over- or under-fitting.
3G	Evaluation	Test accuracy, per-class precision/recall/F1, a confusion matrix. Name the two most-confused class pairs and explain *why* they get confused.
3H	Predictions	A 5 × 5 grid of test images showing true label, predicted label and confidence (wrong ones in red); the 10 most confident wrong predictions.
3I	Inference on new images	Save the best model. Write `predict_image(path)`: load any JPG/PNG photo of clothing, convert to 28 × 28 grayscale, invert if needed, normalise, and return the top-3 classes with probabilities. **You will run this live in the viva on a photo the panel provides.**
3J	Comparison & reflection	A table (model, parameters, training time, validation accuracy, test accuracy). Explain why the CNN beats the MLP, the effect of augmentation, and what you would try next.

Bonus (up to +3 marks, total for Task 3 capped at 20)
CIFAR-10 (`tf.keras.datasets.cifar10` / `torchvision.datasets.CIFAR10`) with transfer learning (e.g. MobileNetV2 or ResNet18) reaching ≥ 85 % test accuracy, **or** a Grad-CAM / saliency visualisation that explains Model 3's decisions on 6 test images.

7. Submission

Submit one zip file named `FirstName_LastName_DS_Trainer_Assessment.zip` containing the git repository, with this structure:

```
FirstName_LastName_DS_Trainer_Assessment/
├── README.md                # setup, exact run commands, assumptions, time spent per task
├── requirements.txt         # pinned versions
├── DECLARATION.pdf          # signed Appendix C
├── sectionA/sectionA_solutions.py
├── task1_analysis/
│   ├── src/ (load.py, clean.py, validate.py, analysis.py)
│   ├── notebooks/task1_analysis.ipynb
│   ├── build_report.py
│   └── outputs/ (UrbanCart_Analysis_Report.pdf, data_quality_log.csv, cleaning_funnel.csv, cleaned/)
├── task2_ml/
│   ├── src/ (cleaning.py, features.py, train.py)
│   ├── notebooks/task2_modelling.ipynb
│   ├── predict.py
│   ├── tests/test_pipeline.py
│   ├── models/final_model.joblib
│   └── outputs/ (predictions_loan_test.csv, scenario_predictions.csv, model_comparison.csv)
├── task3_dl/
│   ├── notebooks/task3_deep_learning.ipynb
│   ├── predict_image.py
│   ├── models/ (best_model.keras or best_model.pt)
│   └── outputs/ (curves, confusion matrix, prediction grids, comparison.csv)
└── .git/                    # full, unsquashed history
```

Pre-submission checklist

- ☐ All notebooks restarted and run top-to-bottom with outputs. ☐ `pytest` passes. ☐ `python build_report.py` regenerates the PDF.
- ☐ `predictions_loan_test.csv` has exactly 8,000 rows and the 3 required columns. ☐ No absolute paths (use relative paths / pathlib).
- ☐ Datasets are **not** included in the zip (the evaluator has them). ☐ No AI tools were used. ☐ Declaration signed.

8. How you will be evaluated

Section	Marks	Main criteria
A · Logic	20	Correct on hidden tests, follows the restrictions, efficiency, edge cases, clarity of A11 explanations
1 · Analysis	25	Completeness of the audit and cleaning, correctness of numbers, depth of insight, pandas idiom, quality of the automated report
2 · ML	25	Leak-free methodology, cleaning quality, breadth and fairness of model comparison, threshold and cost reasoning, tests, hold-out score
3 · DL	20	Correct protocol (validation vs test), models meeting targets, training diagnostics, error analysis, working inference
V · Viva	10	Ownership of the code, live modification, clarity and correctness of the 10-minute teach-back

Across all sections, assessors also reward reproducibility, clean modular code, sensible naming, clear markdown narration (teaching quality), and honest discussion of limitations.

Appendix A — Task 1 data dictionary

The descriptions below give the **intended** meaning of each field. The actual contents may not match these descriptions, and you must check them.

sales_transactions.csv

Column	Intended meaning
Order ID	Order identifier. An order with several products has several lines.
Line No	Line number within the order
Order_Date	Date and time the order was placed (IST)
Customer_ID	Customer key → customers.csv
Product Code	Product key → products.json
Qty	Units purchased on this line
Unit Price (INR)	Selling price per unit before discount, in Indian Rupees
Discount	Discount applied to the line
Shipping Fee	Shipping charged on the line (INR)
Payment Method	UPI, Credit Card, Debit Card, Cash on Delivery, Net Banking, Wallet
STATUS	Delivered, Shipped, Cancelled or Returned
Delivery City	City the order was delivered to → city_master.csv
delivery_days	Days from order to delivery (only meaningful for delivered or returned lines)
Rating	Customer rating on a 1–5 scale (optional; many customers do not rate)
Channel	Mobile App, Website or Phone Order
Coupon	Coupon code used, if any

customers.csv

Column	Intended meaning
customer_id	Unique customer key
full_name, gender, dob	Customer name; Male / Female / Other; date of birth
email, phone	Contact details (Indian 10-digit mobile numbers)
home_city	Registered city
signup_date, last_updated	Date the account was created; date the CRM record was last modified
loyalty_tier	Bronze / Silver / Gold / Platinum
acquisition_channel	Marketing channel that acquired the customer

products.json and city_master.csv

Column	Intended meaning
product_code	Unique product key
product_name, brand	Descriptive attributes
category, sub_category	8 categories: Electronics, Fashion, Home & Kitchen, Beauty & Personal Care, Grocery & Gourmet, Sports & Fitness, Books, Toys & Baby
list_price, cost_price	Catalogue selling price and purchase cost per unit (INR)
launch_date, is_active	Date the product was listed; whether the catalogue record is current
city, state, tier, region	city_master.csv: canonical city name, state, tier (1 = metro, 2, 3) and region

Appendix B — Task 2 data dictionary

Column	Intended meaning
applicant_id	Application identifier
application_date	Date the application was submitted
age, gender, marital_status, dependents	Applicant demographics
education	Highest education level
employment_type	Salaried / Self-Employed / Business Owner / Freelancer / Retired
employer_category	For salaried applicants: Government / MNC / Private Ltd / Startup
years_at_current_job	Tenure with current employer or business
annual_income	Gross annual income (INR)
city_tier	1 = metro, 2, 3 = smaller towns
residence_type, years_at_residence	Housing status and tenure
loan_purpose	Stated purpose of the loan
loan_amount	Requested principal (INR)
loan_term_months	Repayment tenure
interest_rate	Annual interest rate offered
credit_score	Bureau score (300–900). Applicants with no credit history have no score.
num_credit_lines	Open credit accounts
existing_monthly_emi	Current EMIs on other loans (INR / month)
late_payments_last_2y	Late payments reported by the bureau in the last 24 months
credit_utilization	Revolving credit used ÷ credit limit
hard_inquiries_6m	Credit enquiries in the last 6 months
bank_relationship_months	Months the applicant has banked with FinTrust
has_cosigner, has_collateral	Whether a co-signer / collateral was offered
application_channel	Branch / Online / DSA Agent / Partner App
branch_code	Originating branch
debt_to_income	Total monthly obligations ÷ monthly income, as captured by the origination system
recovery_calls_made	Number of calls made by the recovery team on this loan account
country	Country of the applicant
loan_amount_lakhs	Loan amount in lakhs
record_created_by	System or user that created the record
defaulted	**Target.** 1 if the loan went 90+ days past due within 12 months of disbursal (train file only)

Appendix C — Declaration of original work

I, _____, declare that:

1. All code, analysis, reports and explanations submitted for this assessment are entirely my own work.
2. I did not use ChatGPT, Claude, Gemini, Copilot, Cursor, Codeium, Colab AI or any other AI or AI-assisted tool at any stage.
3. I did not receive help from any other person, and I did not copy code from public notebooks, repositories or websites.
4. I can explain and modify any part of my submission on request, and I accept that failing to do so may void my submission.
5. I understand that any breach of these rules leads to disqualification.

Signature		Date	